# Data Structures and Algorithms
# Lecture 3 – Queues

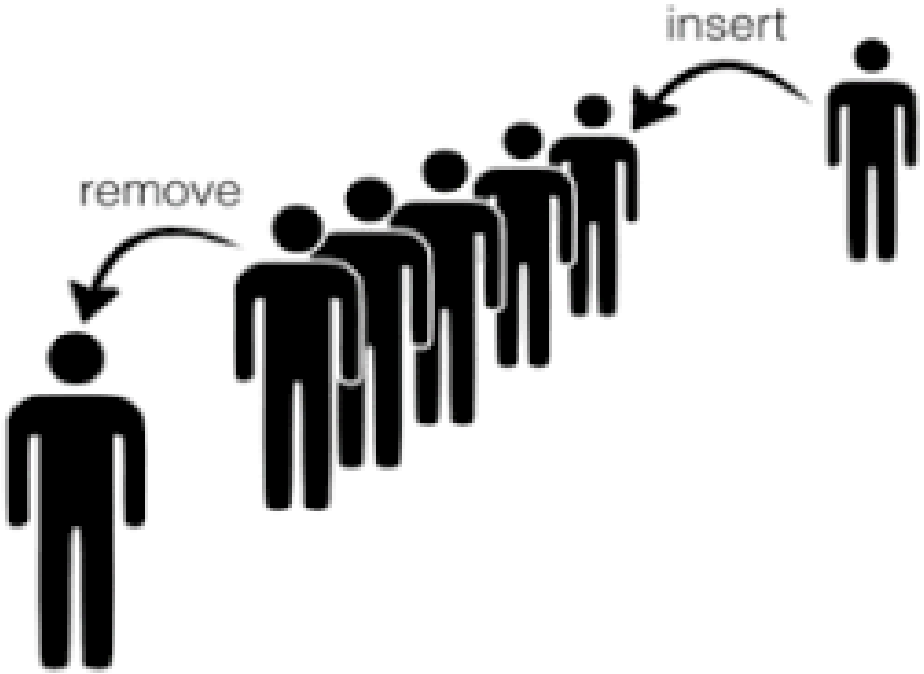
**M.G.Noel A.S Fernando (PhD)**

**Professor**

**UCSC**

# What is a Queue?

# Queues



# Queues

- A queue is another special kind of list, where items are inserted at one end called the rear(back) and deleted at the other end called the front.
- Another name for a queue is a “FIFO” or “First-in-first-out” list

# The Queue Data Structure

- A queue is used in computing in much the same way as it is used in every day life: allow a sequence of items to be processed on a **first-come-first-served basis.**

- In most computer installations, for example, one printer is connected to several different several machines. So that more than one user can submit printing jobs to the same printer, it maintains a queue.

# Queues in computer science

- Operating systems:
  - queue of print jobs to send to the printer
  - queue of programs / processes to be run
  - queue of network data packets to send

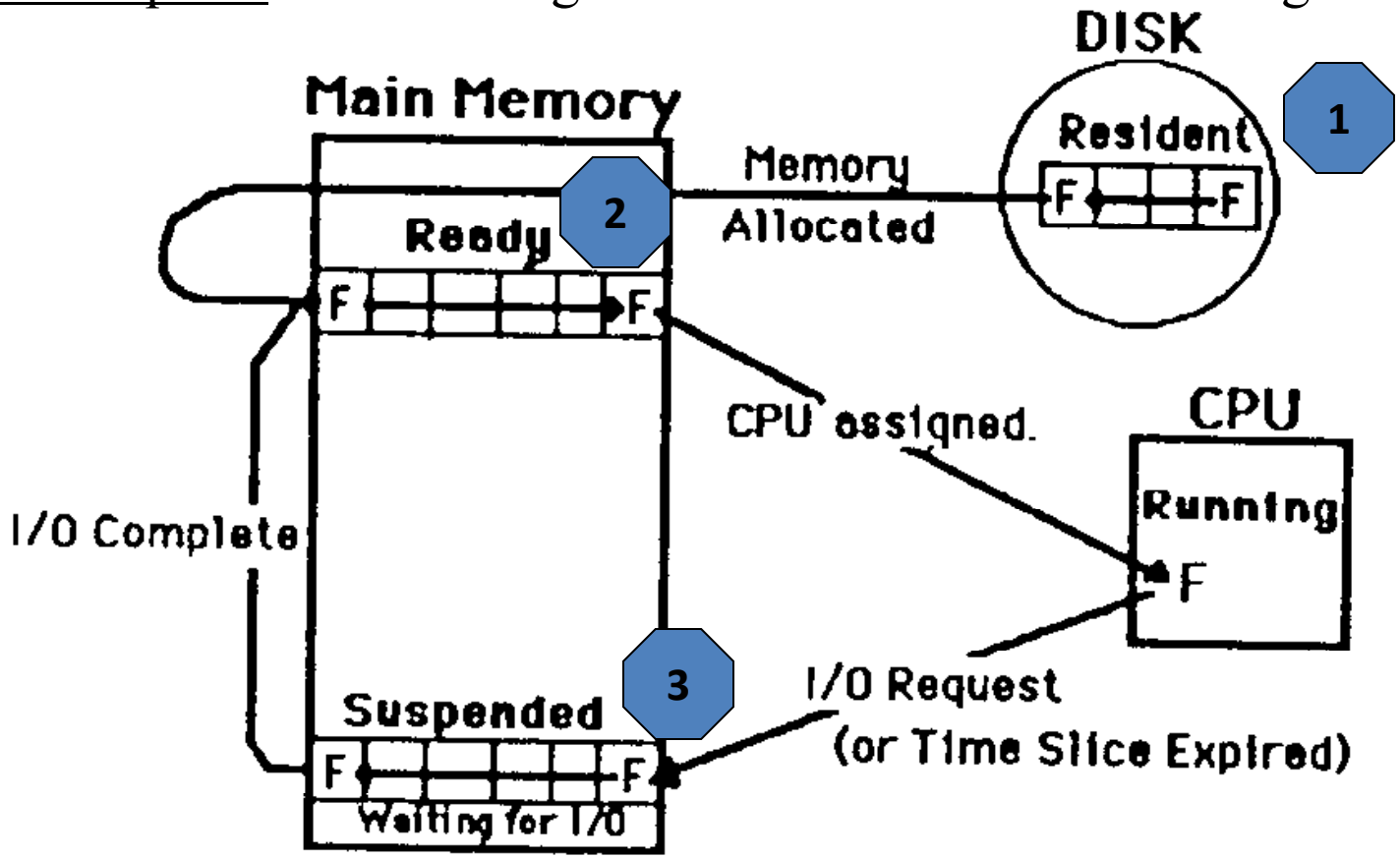
- Programming:
  - modeling a line of customers or clients
  - storing a queue of computations to be performed in order

- Real world examples:
  - people on an escalator or waiting in a line
  - cars at a gas station (or on an assembly line)

# Other Queue examples :

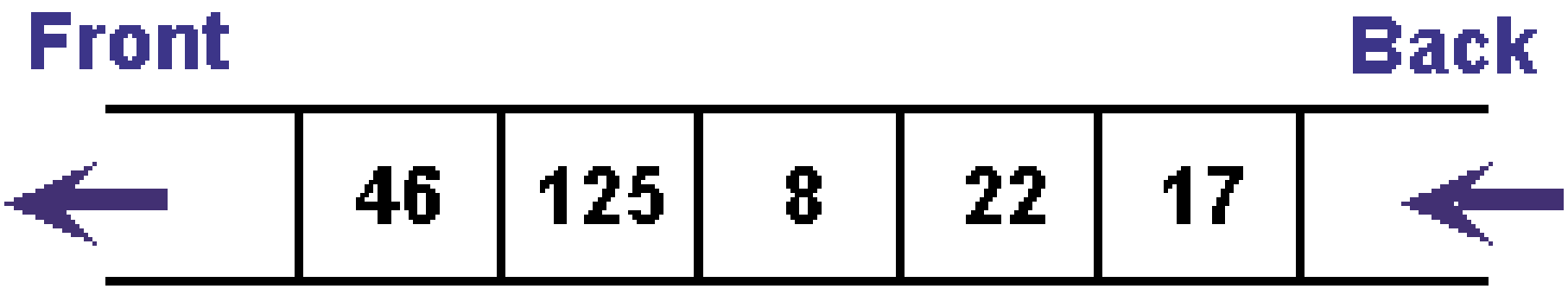
Example: 3- State Transition Diagram (O/S)

1. Resident queue: On disk, waiting for memory
2. Ready queue: In memory — has everything it needs to run, except the CPU
3. Suspended queue: Waiting for I/O transfer or to be reassigned the CPU



# Abstract data type: Queue

- **queue**: a more restricted List with the following constraints:
  - elements are stored by order of insertion from *front* to *back*
  - items can only be added to the *back* of the queue
  - only the *front* element can be accessed or removed



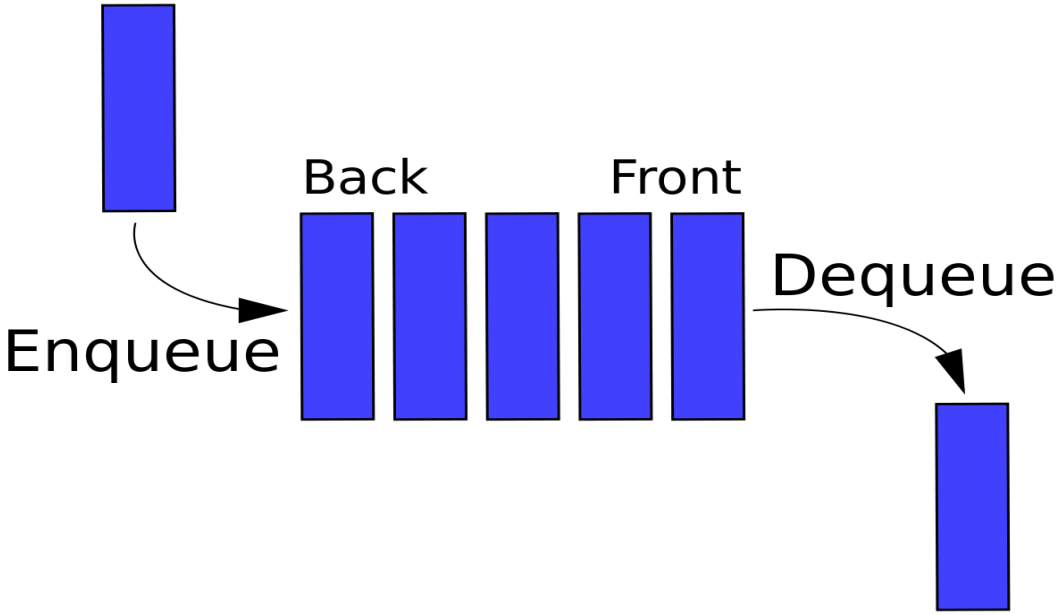

Items enter queue at back and leave from front.

# ARRAY REPRESENTATION OF QUEUES

- The operations for a queue are analogues to those for a stack, the difference is that the insertions go at the end of the list, rather than the beginning.
- Queues can be easily represented using linear arrays.
- As stated earlier, every queue has front and rear variables that point to the position from where deletions and insertions can be done, respectively.

# Main Operations on a queue

- *enqueue*: which inserts an element at the end of the queue (rear).
- *dequeue*: which deletes an element at the start of the queue (front)



# Main Operations on a queue

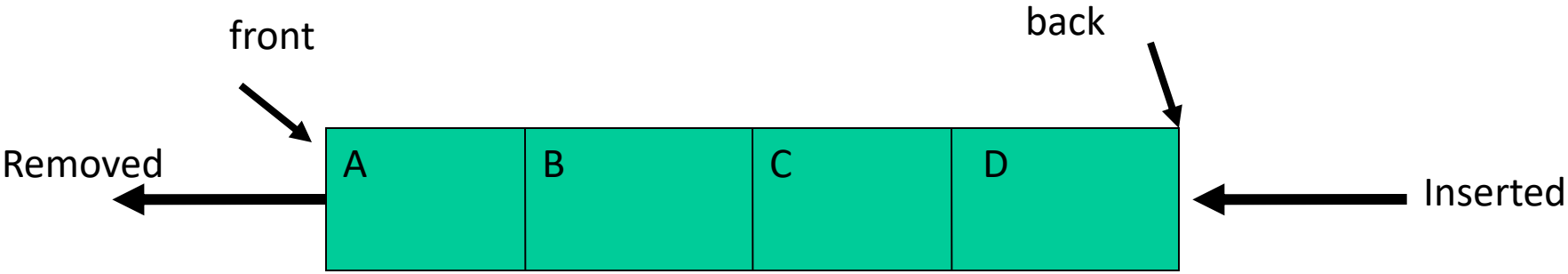
- Following are the basic operations supported by an array.
- **Traverse** – print all the array elements one by one.
- **Insertion** – Adds an element at the given index.
- **Deletion** – Deletes an element at the given index.
- **Search** – Searches an element using the given index or by the value.
- **Update** – Updates an element at the given index.
- **Display** – Displays the contents of the array.

# Components of a queue

- **Front** is a variable which refers to first position in queue.
- **Rear** is a variable which refers to last position in queue.
- **Element** is component which has data.
- **MaxQueue** is variable that describes maximum number of elements in a queue.

# Queue Definition

- A queue is an ordered collection of items from which items may be deleted at one end ( called front of the queue) and into which items may be inserted at the other end (called the rear of the queue)

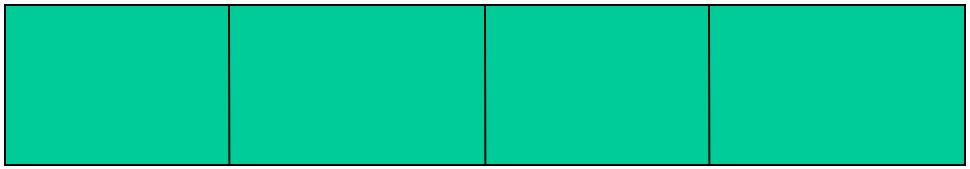


# Queue Implementation

- There are two variables
- i.e. front and rear, that are implemented in the case of every queue.
- Front and rear variables point to the position from where insertions and deletions are performed in a queue.
- Initially, the value of front and queue is -1 which represents an empty queue.

# Queue initialisation

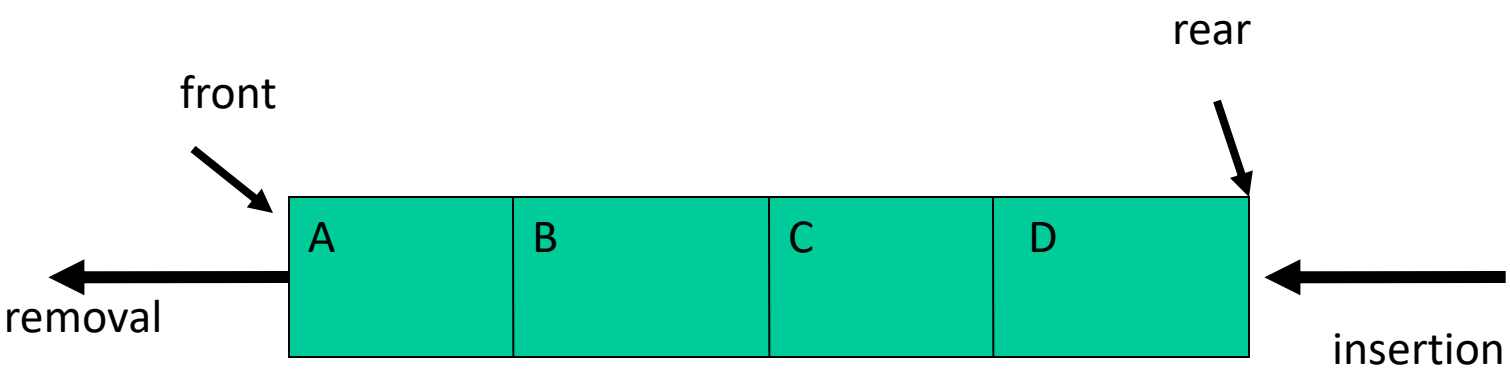
- For an empty queue



Front, Rear

If FRONT = –1 and REAR = –1, it means there is no element in the queue

# Basic Array implementation of queues (general situation)

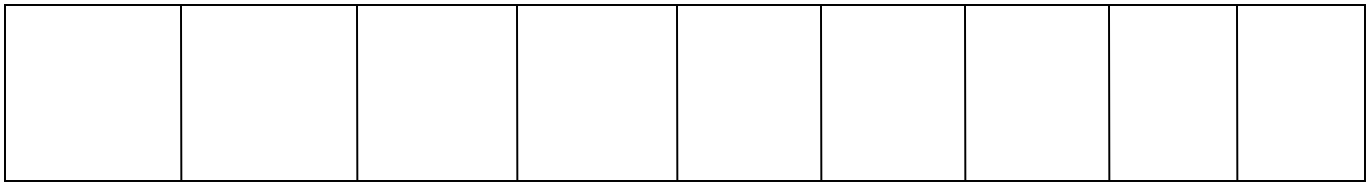



After insertion of A,B,C and D

Front=0 and rear=3

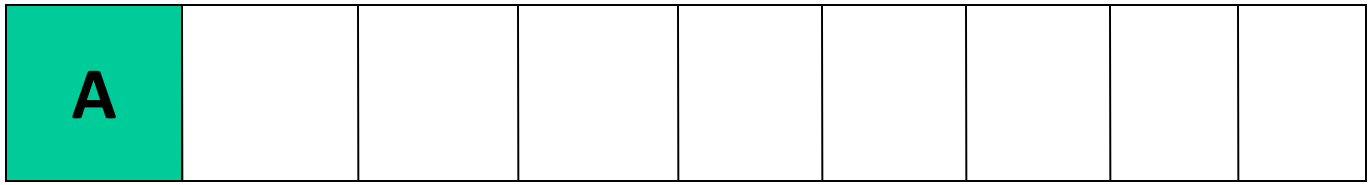
# Array implementation of queues

**Eg:**



If FRONT = –1 and REAR = –1, it means there is no element in the queue

**<u>Enqueue(A)</u>**




FRONT = 0 and REAR = 0

## Enqueue(B)

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| A | B | | | | | | | |

FRONT = 0 and REAR = 1

## Dequeue()

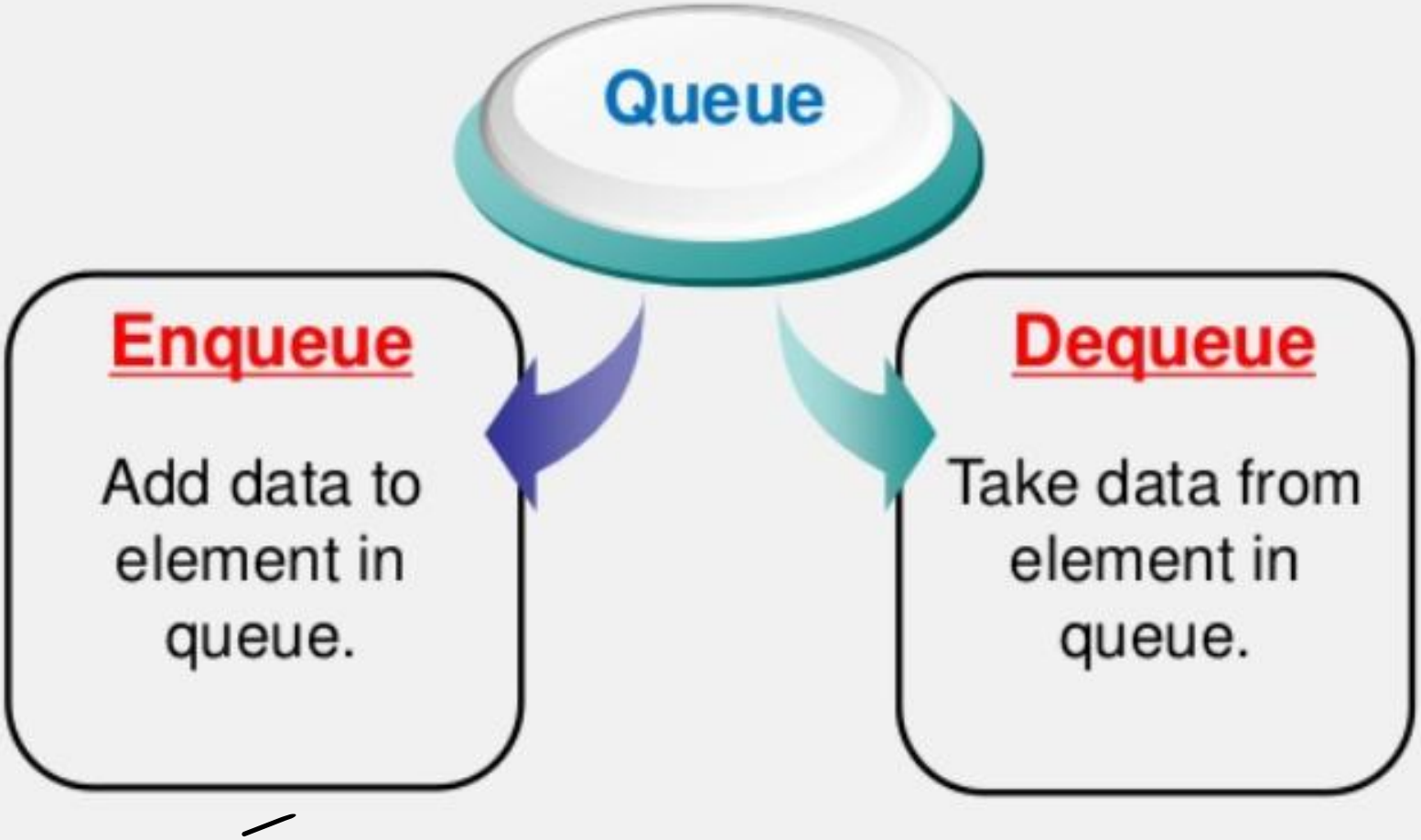
| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | B | | | | | | | |

FRONT=1, REAR=1

# Array implementation of queues Cont…….

- **B**efore inserting an element in a queue, we must check for overflow conditions.
- An overflow will occur when we try to insert an element into a queue that is already full(Rear = Max – 1)
- Before deleting an element from a queue, we must check for underflow conditions.
- An underflow condition occurs when we try to delete an element from a queue that is already empty.
- If Front = –1 and Rear = –1, it means there is no element in the queue.

# Main Operation



# Enqueue

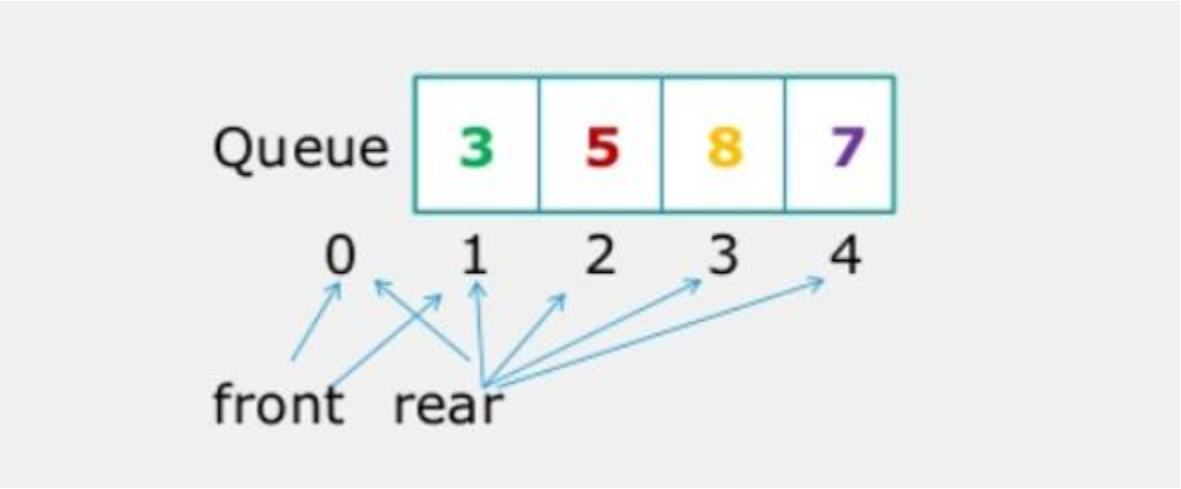
- Steps in queue operations:
- Queue can be added when it’s  not full.
- If queue is empty, then front and rear is -1

# Enqueue

Steps in enqueue operation:

- Queue can be added when it's not full
- If queue is empty then front and rear is added by 1. For the contrary, rear is added by 1.
- Queue element, which was refered by rear pointer, is filled with new data.

# Enqueue




if queue is empty front=rear=-1

Enqueue(3) , front=0,rear=0

Enqueue(5) , front=0,rear=1

Enqueue(8) , front=0,rear=2

Enqueue(7) , front=0,rear=3

Enqueue(2) , front=0,rear=4, No enqueue, queue is full

d

# Question 1

- Write a pseudocode algorithm to insert an element into a queue (Enqueue).
  Then convert the pseudocode algorithm into C code.

# Question 2

- Write a pseudocode algorithm to delete an element from a queue (Dequeue).
  Then convert the pseudocode algorithm into C code.

# Queue implementation (Cont..)
# isFull & isEmpty

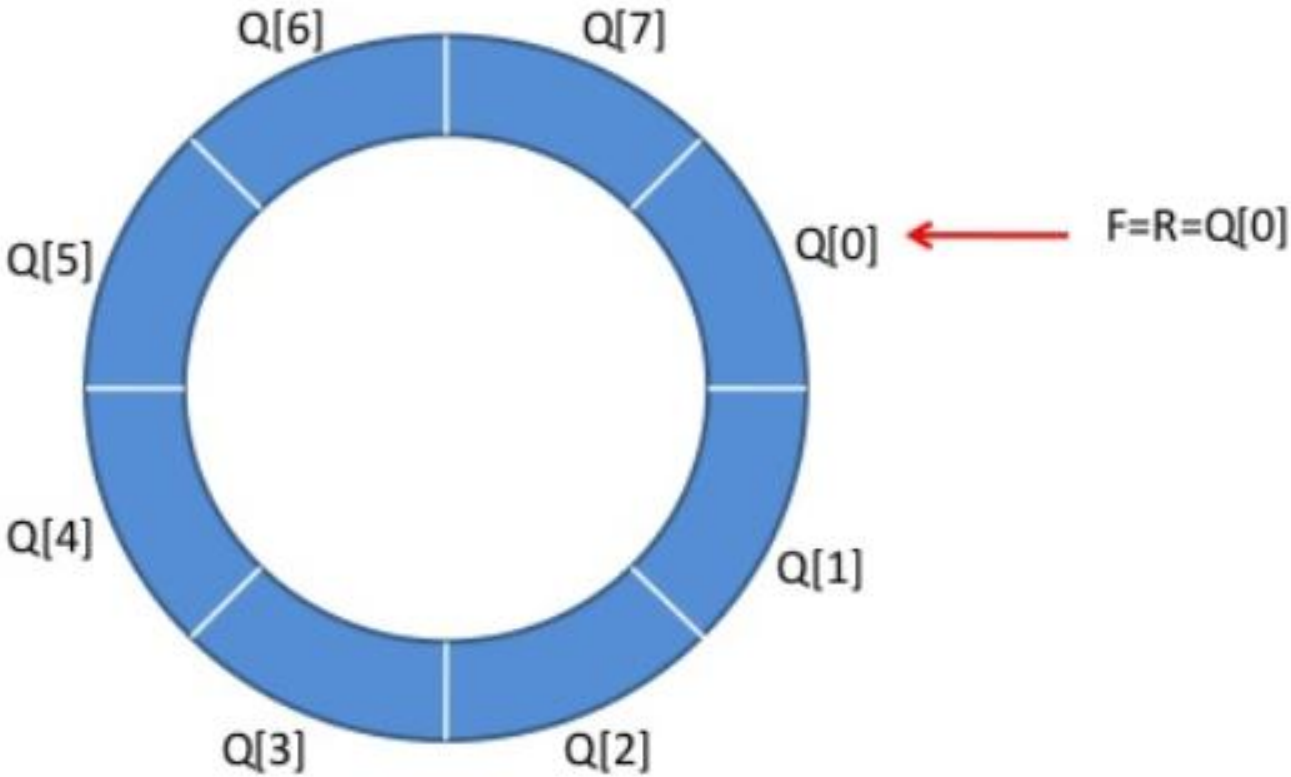
```
boolean isFull() {
 if (front == 0 && rear == SIZE - 1) {
return true;
 }
return false;
}
boolean isEmpty() {
if (front == -1)
return true;
else
return false;
 }
```

# Circular Queue

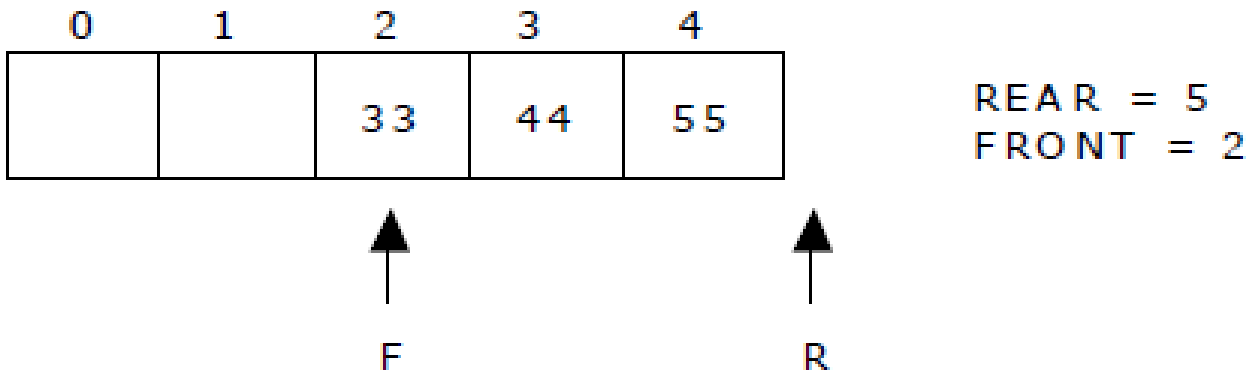
- There are two problems associated with linear queue.
- They are:
- **Time consuming**: linear time to be spent in shifting the elements to the beginning of the queue.
- **Signaling (indicating) queue full**: even if the queue is having vacant position.

# Circular Queue

- e.g.:



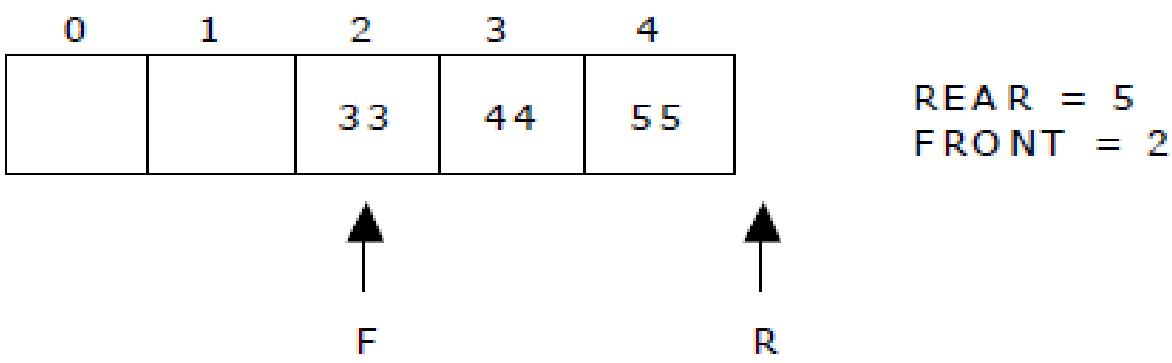
# For example, let us consider a linear queue status as follows:




- If one wants to insert 66 to the queue.
- The rear crossed the maximum size of the queue (i.e., 5).
- There will be queue full signal.
- The queue status is as follows:

# For example, let us consider a linear

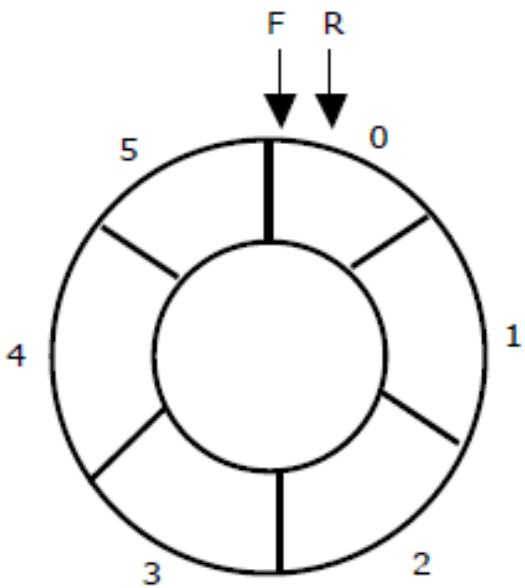
- The queue status is as follows:




- This difficulty can be overcome if we treat queue position with index zero as a position
- That comes after position with index four then we treat the queue as a **circular queue.**

# Representation of Circular Queue

- Let us consider a circular queue, which can hold maximum (MAX) of six elements.
- Initially the queue is empty.

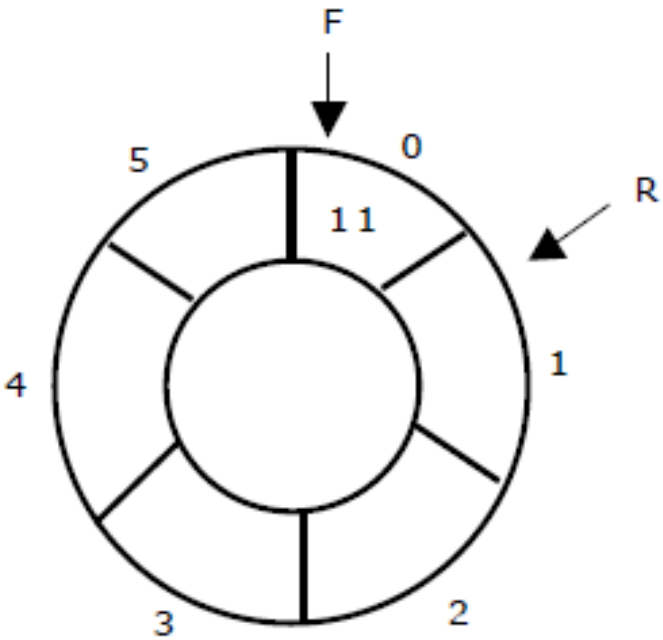



Circular Queue

Queue Empty
MAX = 6
FRONT = REAR = 0
COUNT = 0

# Representation of Circular Queue

- Now, insert 11 to the circular queue. Then circular queue status will be:

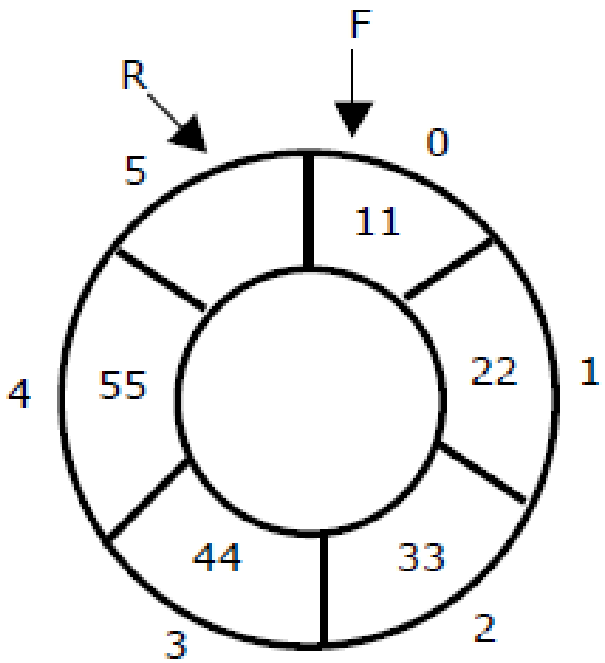



Circular Queue

FRONT = 0
REAR = (REAR + 1) % 6 = 1
COUNT = 1

# Representation of Circular Queue cont..

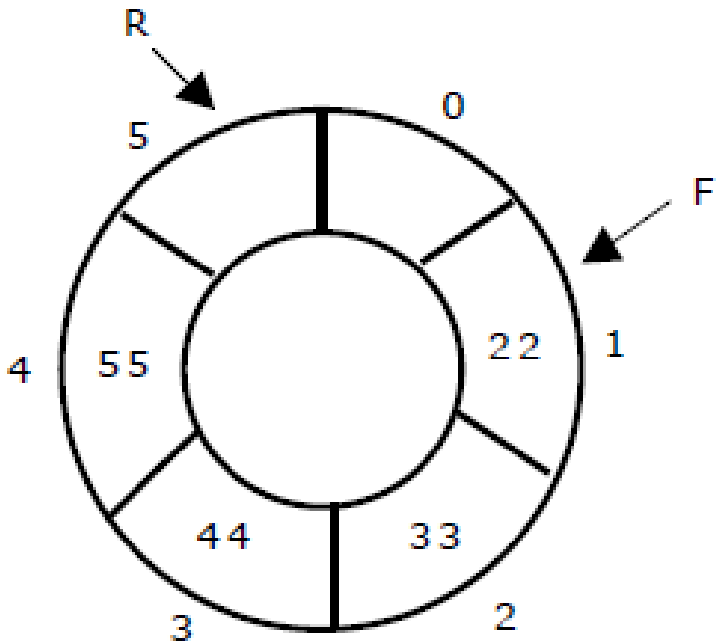
- Insert new elements 22, 33, 44 and 55 into the circular queue




FRONT = 0
REAR = (REAR + 1) % 6 = 5
COUNT = 5

# Representation of Circular Queue cont..

- Now, delete an element. The element deleted is the element at the front of the circular queue. So, 11 is deleted

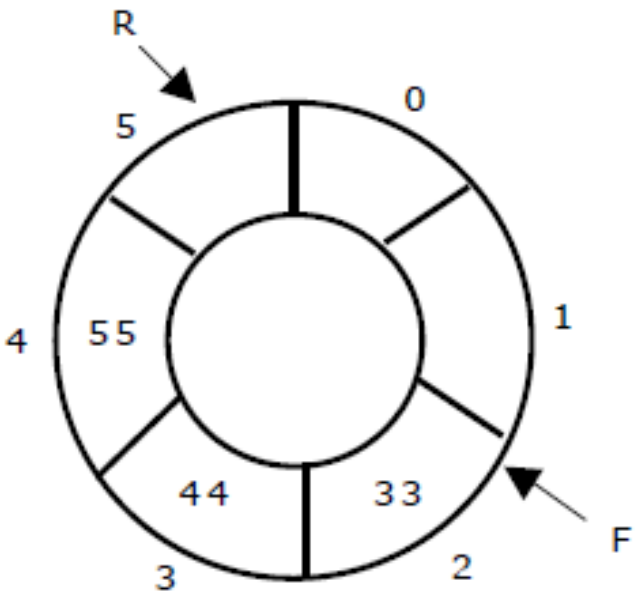



Before delete the element 11,FRONT=0, REAR=5, COUNT=5

```
FRONT = (FRONT + 1) % 6 = 1
REAR = 5
COUNT = COUNT - 1 = 4
```

# Representation of Circular Queue cont..

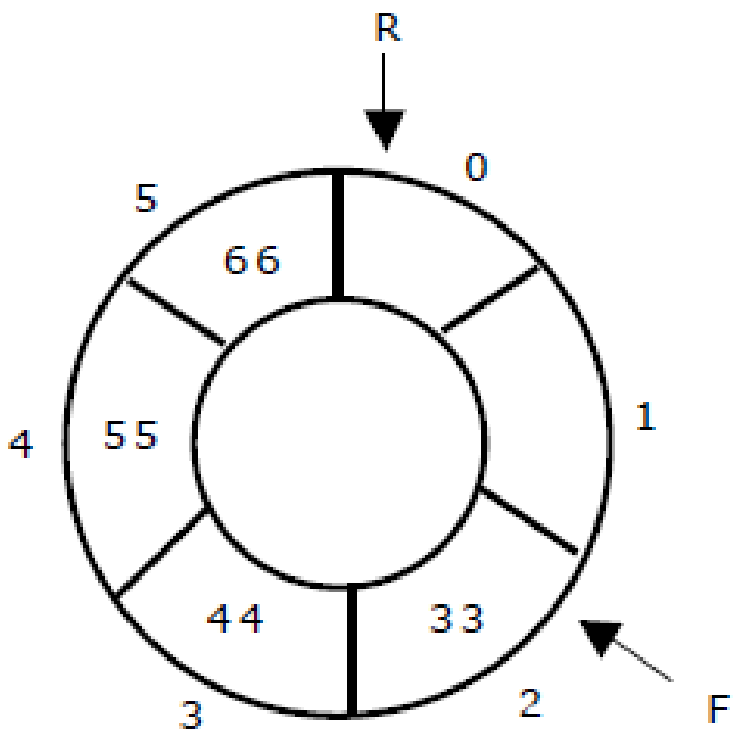
- Again, delete an element. The element to be deleted is always pointed to by the FRONT pointer. So, 22 is deleted.




```
FRONT = (FRONT + 1) % 6 = 2
REAR = 5
COUNT = COUNT - 1 = 3
```

# Representation of Circular Queue cont..

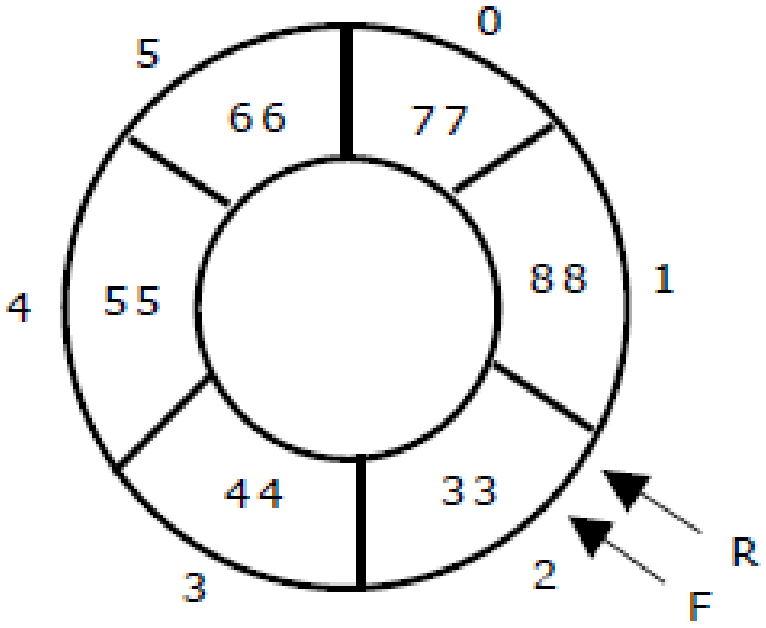
- Again, insert another element 66 to the circular queue.




```
FRONT = 2
REAR = (REAR + 1) % 6 = 0
COUNT = COUNT + 1 = 4
```

# Representation of Circular Queue cont..

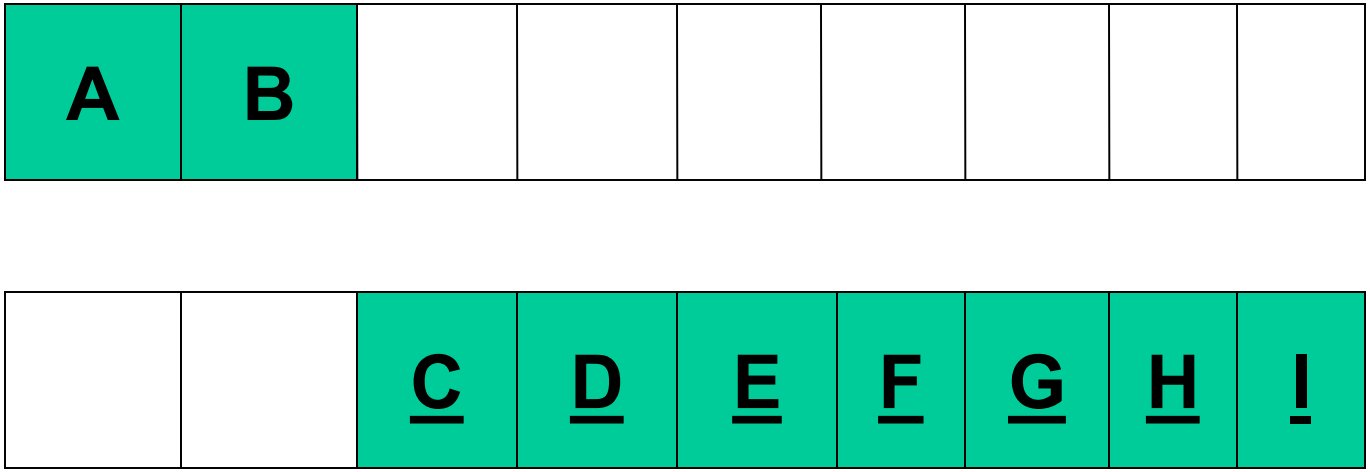
- Now, insert new elements 77 and 88 into the circular queue




FRONT = 2, REAR = 2
REAR = REAR % 6 = 2
COUNT = 6

# Wraparound routine(increment)

```
If (++x==theArray.length)     //maxarray-1
x=0
return x;
}
```



# Wraparound routine(increment)

The increment routine adds 1 to its parameter and returns the new value. Since it implements wraparound, if the result would equal the array size it is wraparound to zero.

# Priority Queue:

- A priority queue is a collection of elements such that each element has been assigned a priority and such that the order in which elements are deleted and processed comes from the following rules:

1. An element of higher priority is processed before any element of lower priority.
2. two elements with same priority are processed according to the order in which they were added to the queue.

# Priority Queue:

- A priority queue is a collection of elements such that each element has been assigned a priority and such that the order in which elements are deleted and processed comes from the following rules:

1. An element of higher priority is processed before any element of lower priority.
2. two elements with same priority are processed according to the order in which they were added to the queue.

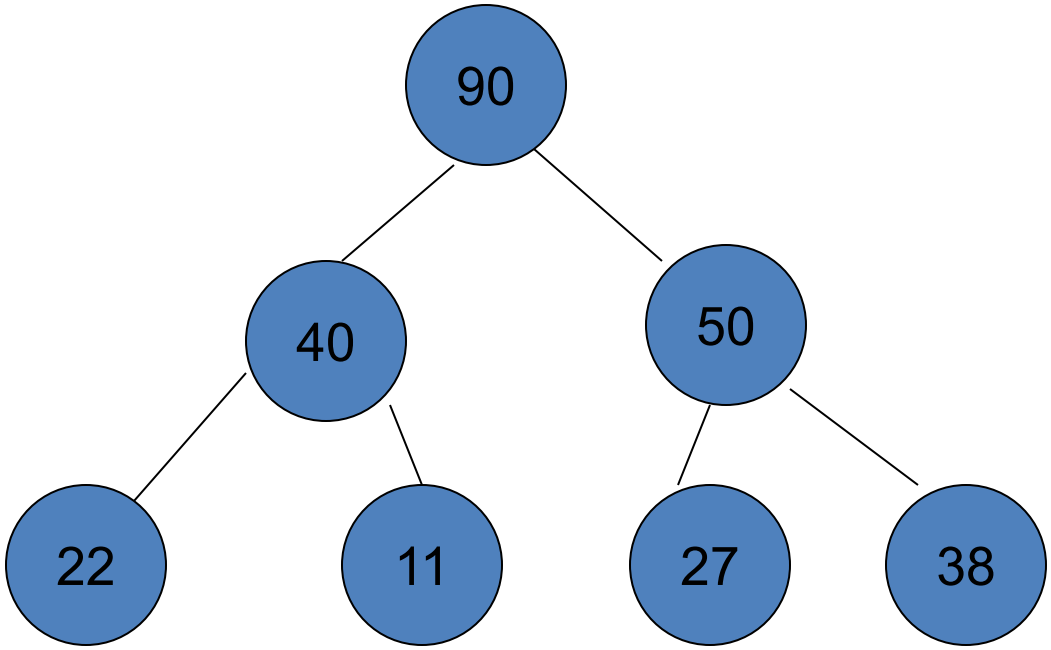
# Priority Queue:

- A prototype of a priority queue is time sharing system:
- programs of high priority are processed first, and programs with the same priority form a standard queue.
- An efficient implementation for the Priority Queue is to use heap, which in turn can be used for sorting purpose called heap sort.

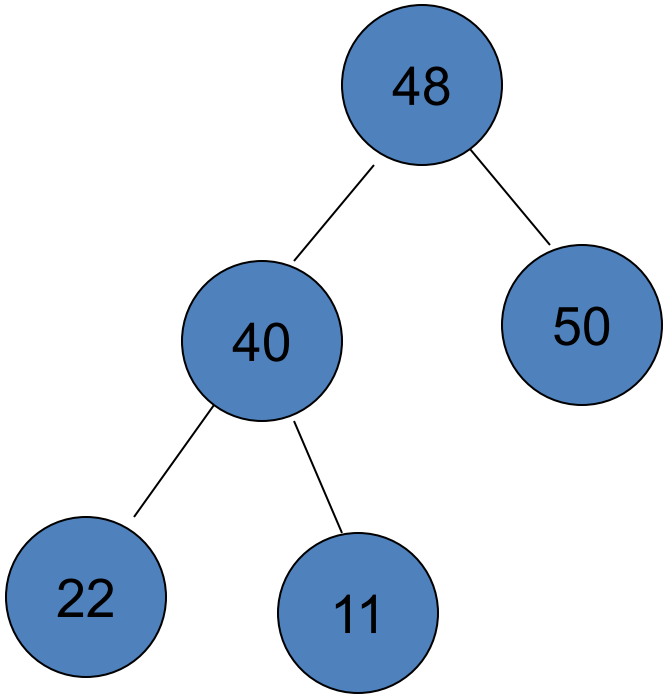
# Heaps, Heap sorts and Priority Queues

- Heap
- A heap of size n is a Binary  tree of n nodes such that the values of each node is >= the value of its children. (if any)

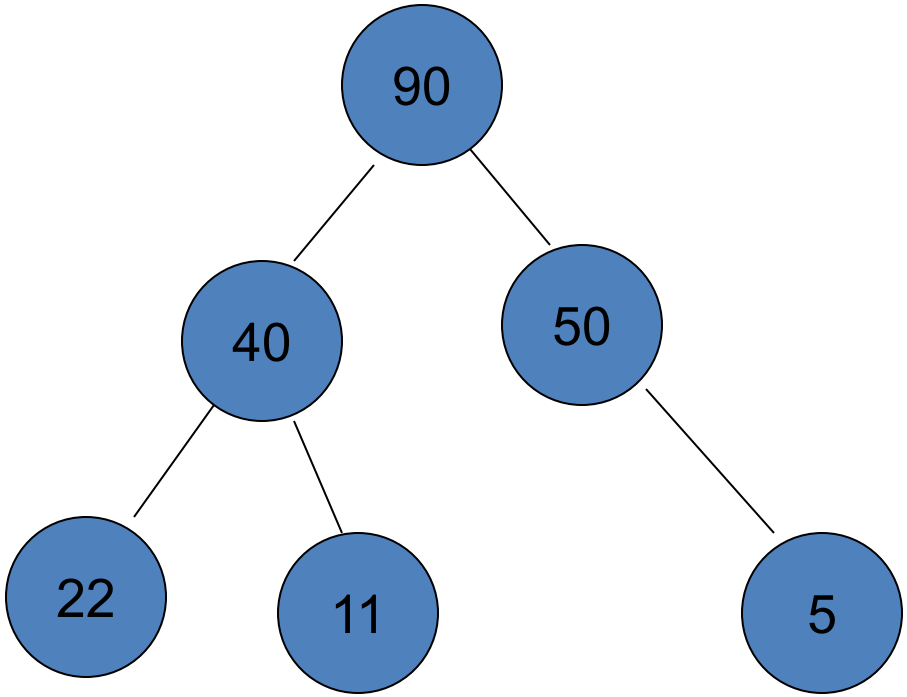
# Examples




A Heap




Not a heap [i.e., 50 <48]




Not a Heap [ Gaps]

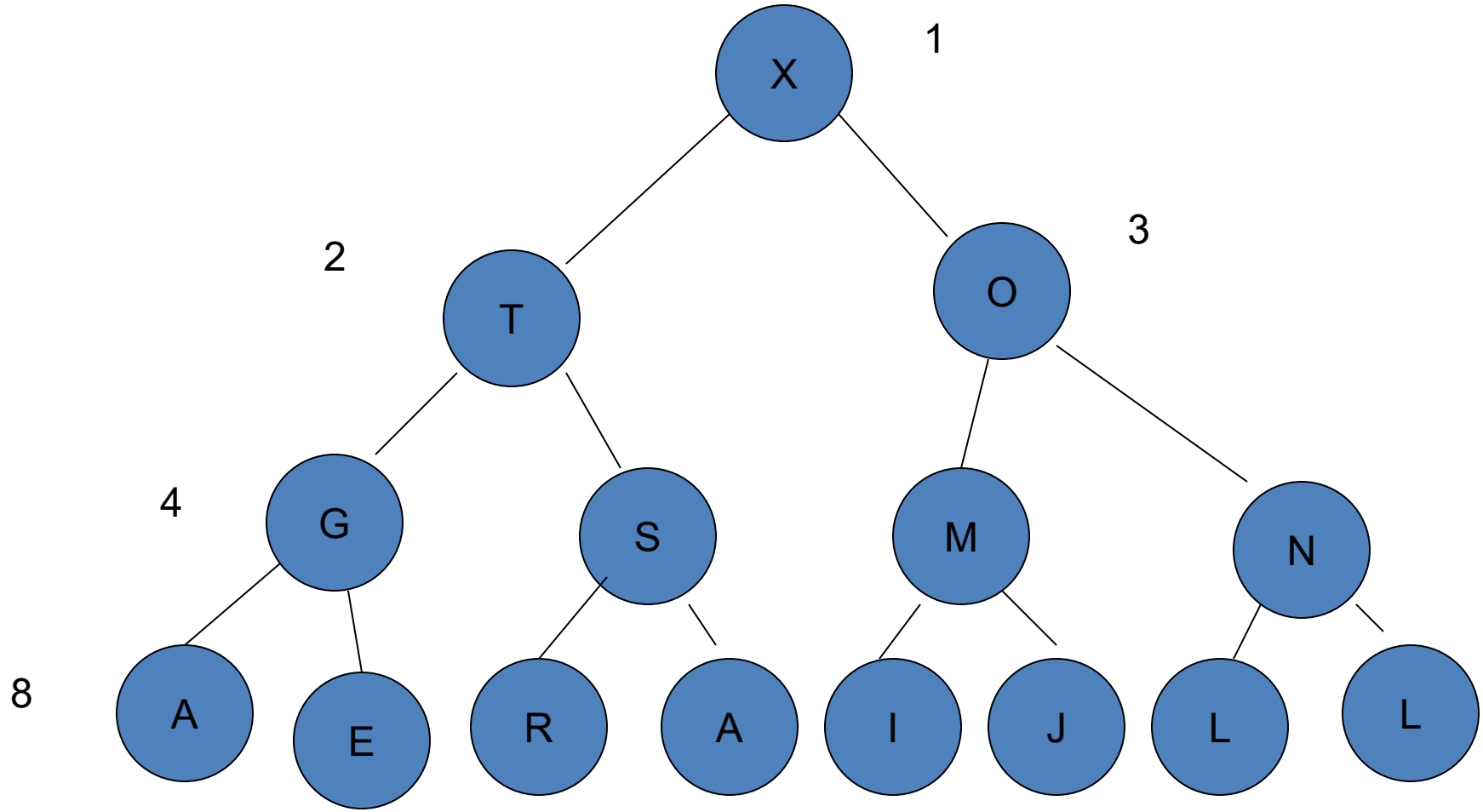
# Properties

- A heap, however, is a special kind of binary tree that leaves no gaps in an array implementation.
- The definition of the heap data type is as follows.

# A heap is a binary tree satisfying the following restrictions.

- All leaves are on adjacent levels.
- All leaves on the lowest level occur at the left of the tree
- All levels above the lowest are completely filled.
- Both children of any node are again heaps.
- The values stored at any node is at least as large as the values in its two children.

# Array representation of a heap

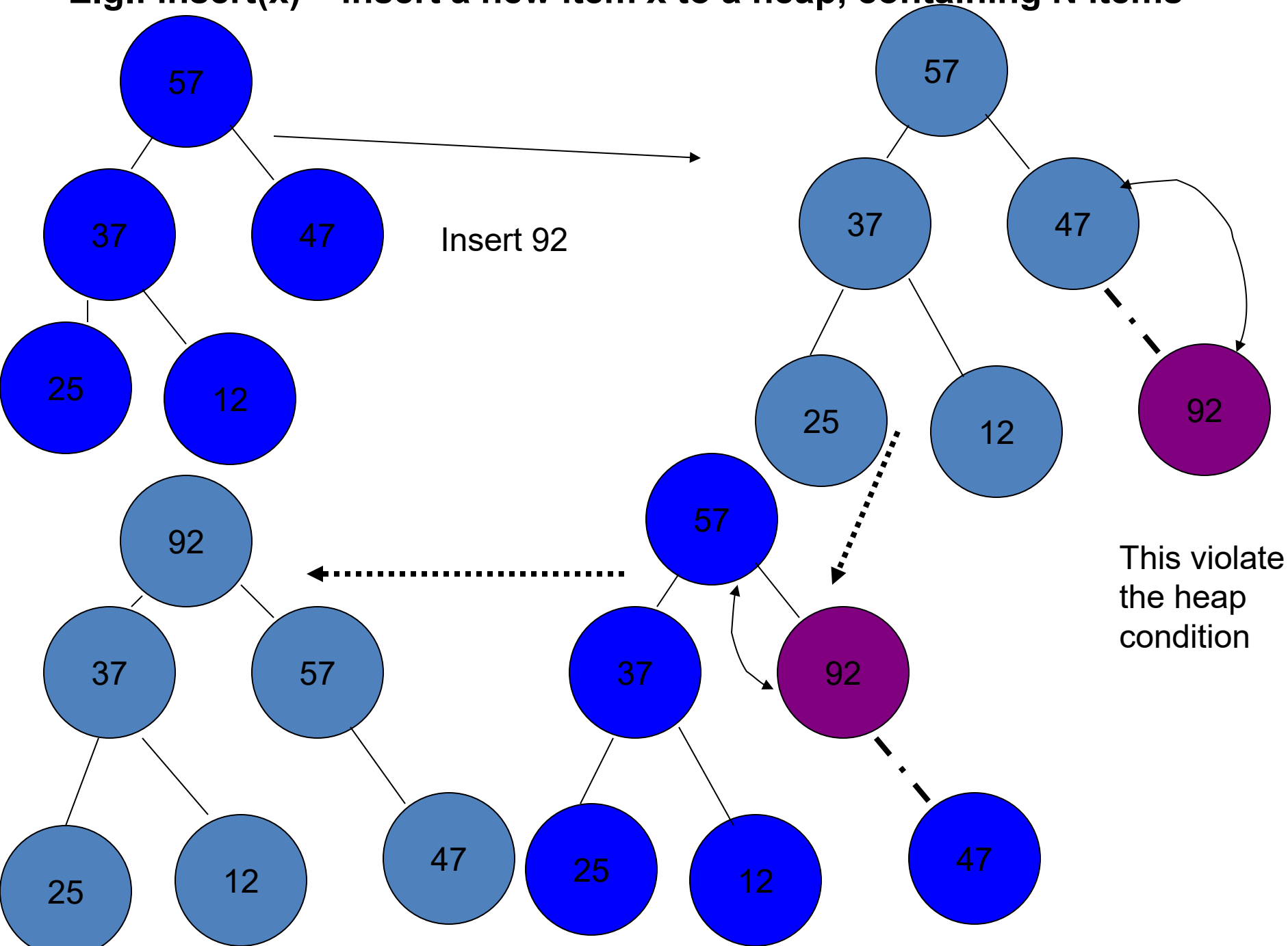



The tree may be easily constructed from the array by noticing that the left child of the k as 2k and right child 2k+1.

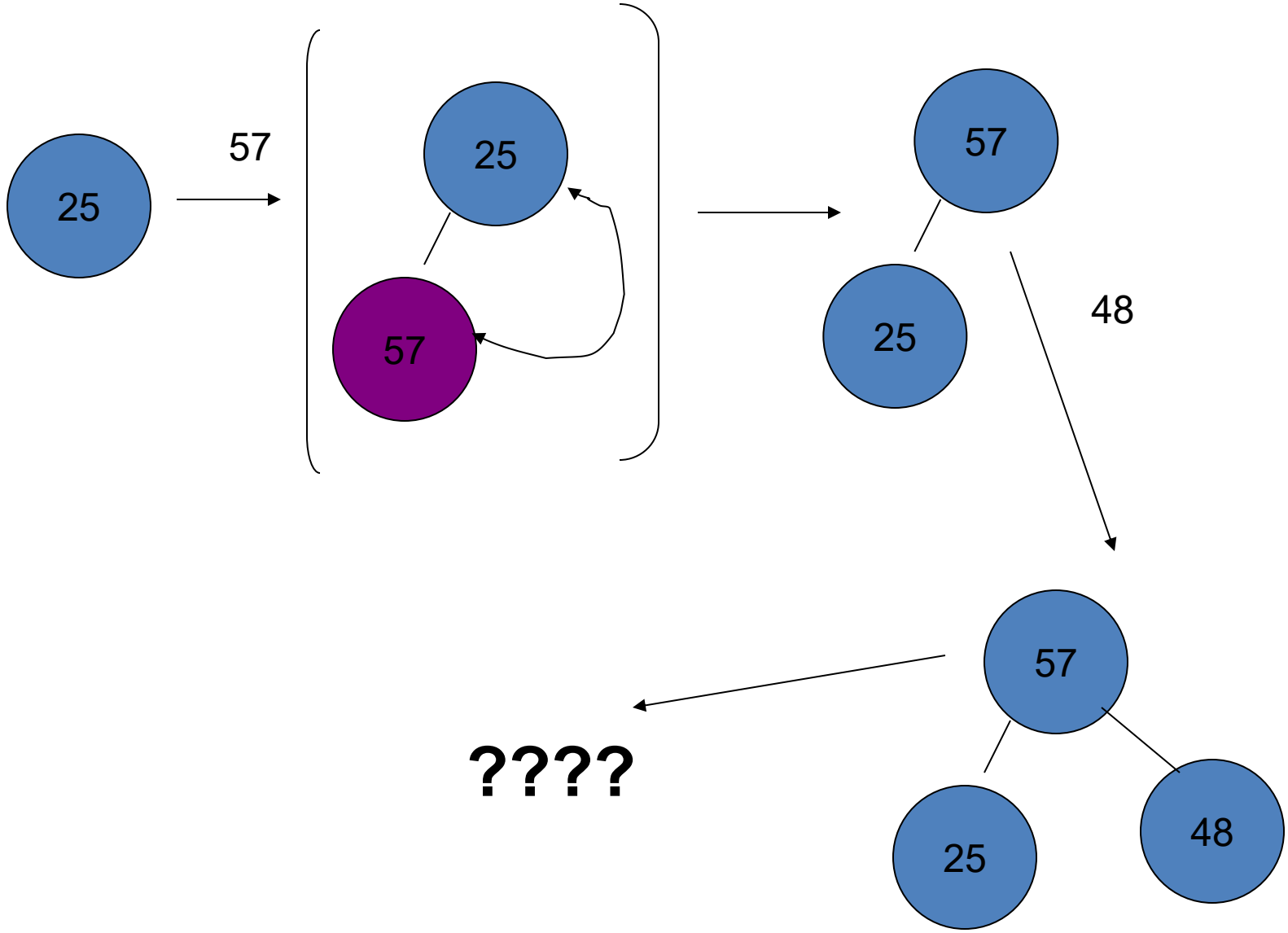
# Algorithms on Heaps

- The algorithms on heap works by first making a simple structural modification which could violate the heap condition, the traveling through the heap modifying it  to ensure that the heap condition is satisfied everywhere.
- Some of the algorithm travel through the heap from bottom to top, others top to bottom.

# E.g.: insert(x) – insert a new item x to a heap, containing N items



Creating a Heap of size 8. The original file is 25,57,48,37,12,92,86,33



- Priority Queue Operations
- We want to build and maintain a data structure containing records with numerical keys ( priorities ) and supporting some of the following operations.
- Construct a priority queue from given N items
- Insert a new item ( with a priority)
- Replace the largest  item with a new item.( unless new item is larger)
- Change the priority of an item.

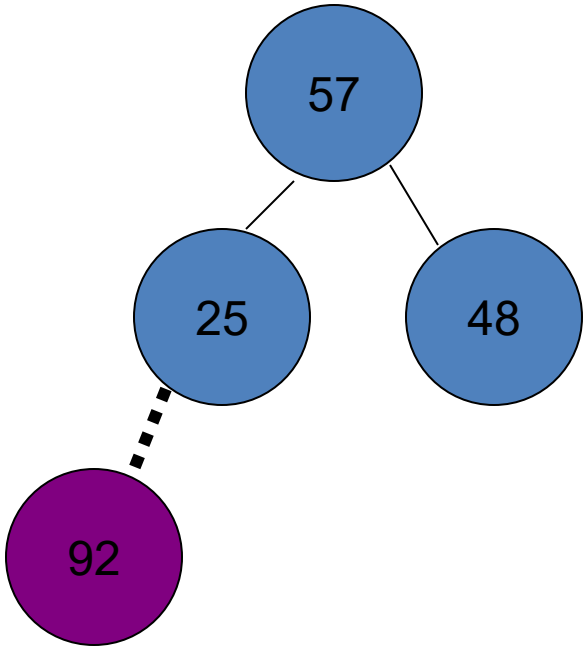
# • Priority Queue Operations

- To be able build a heap, it is necessary, first implement the insert operation.
- This operation  will increase the size of the heap by one.
- N must be incremented.
- Then the record to be inserted is put into A[N] , but this may violate the heap property (ie. New node is grater than its parent ) then the violation can be fixed by exchanging new node with its parent .

- Priority Queue Operations

- The code for this method is straight forward.
- Insert : add a new item to a[n], then upheap[n] fix the heap condition violation at N.

# Pseudo code for insertion




Insert (X)

N++

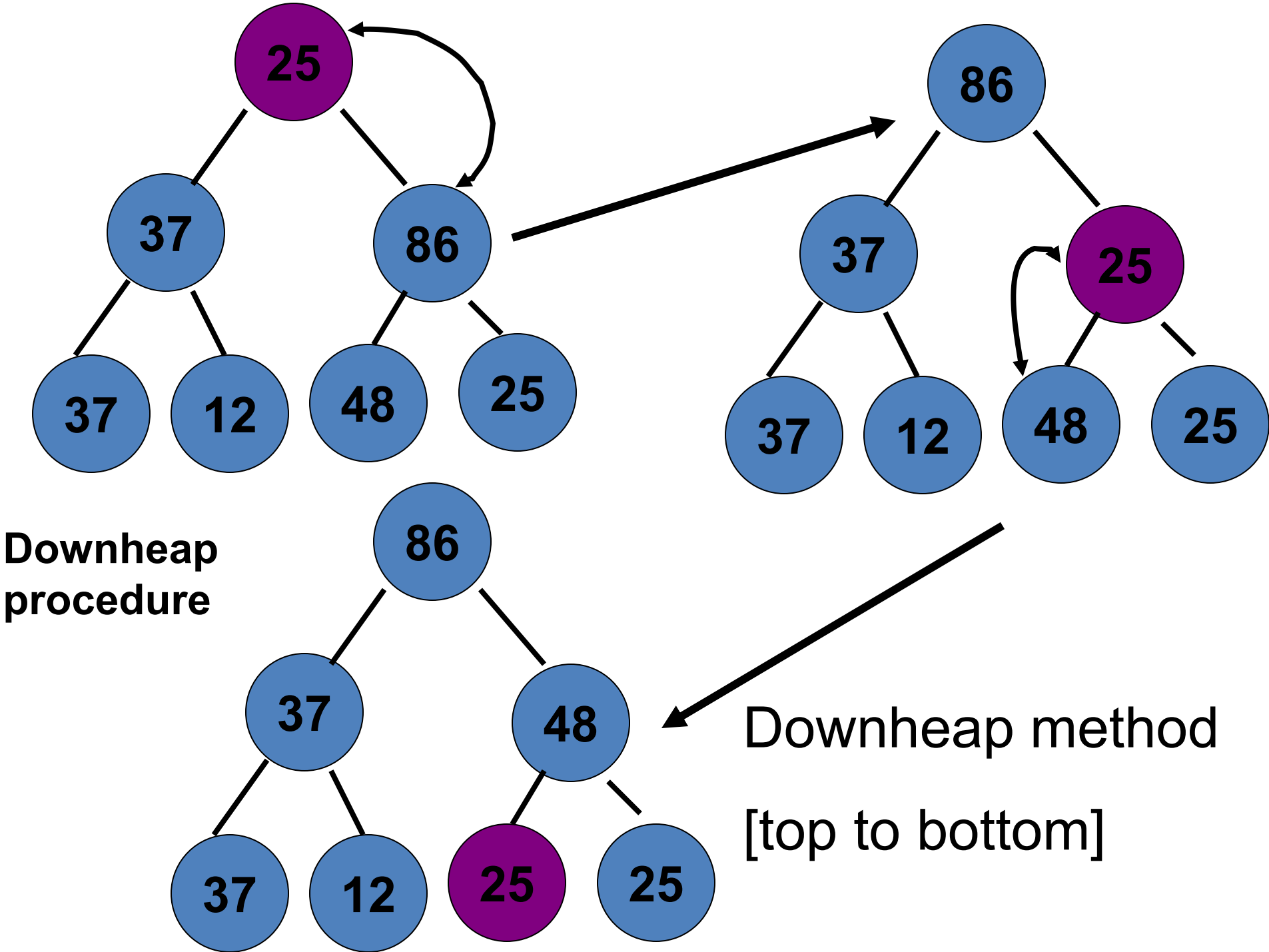
A[N]=x

Upheap{N]

Upheap procedure ??

# Downheap Method

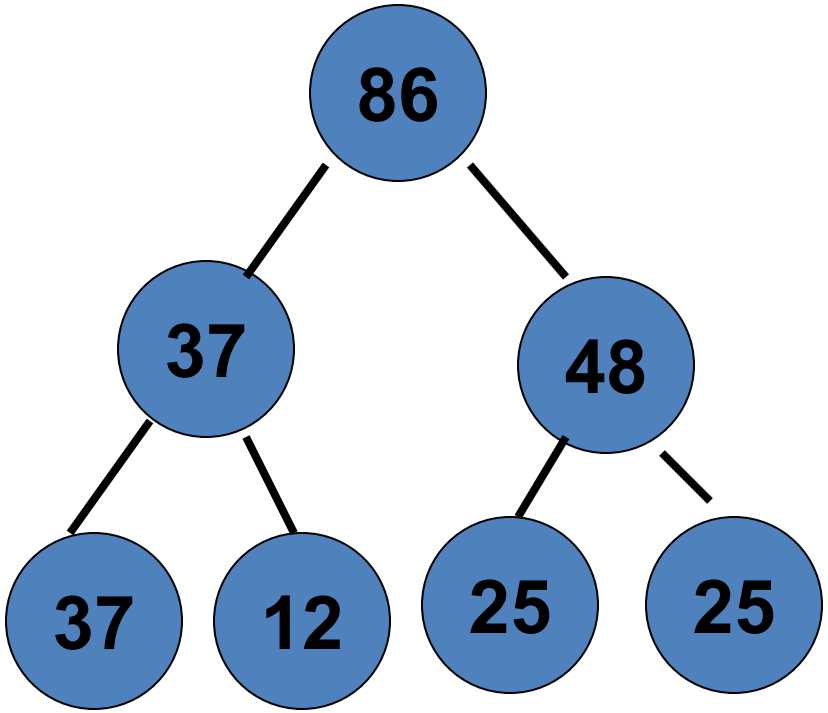
- If the element at node k is smaller than its children, that element is moved down the heap without violating the heap property at any nodes touched.


25
37
86
37
12
48
25
86
37
25
37
12
48
25
Downheap procedure
86
37
48
37
12
25
25
Downheap method
[top to bottom]

# Remove the largest item-pseudo code

Remove the highest priority element ( ) ;

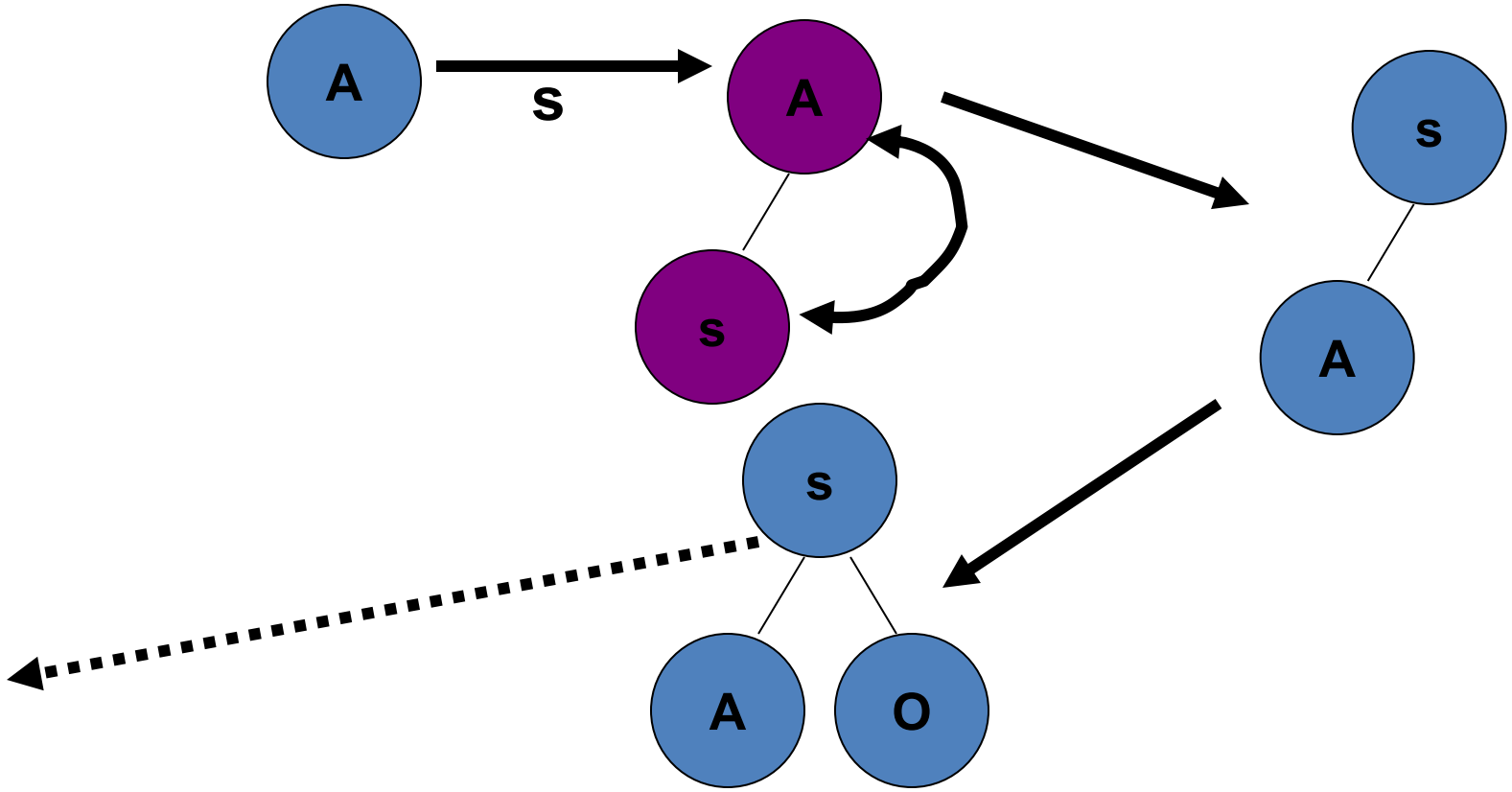
```
return = A[0];
A[0] = A[N-1];
N--;
Upheap[N];
```



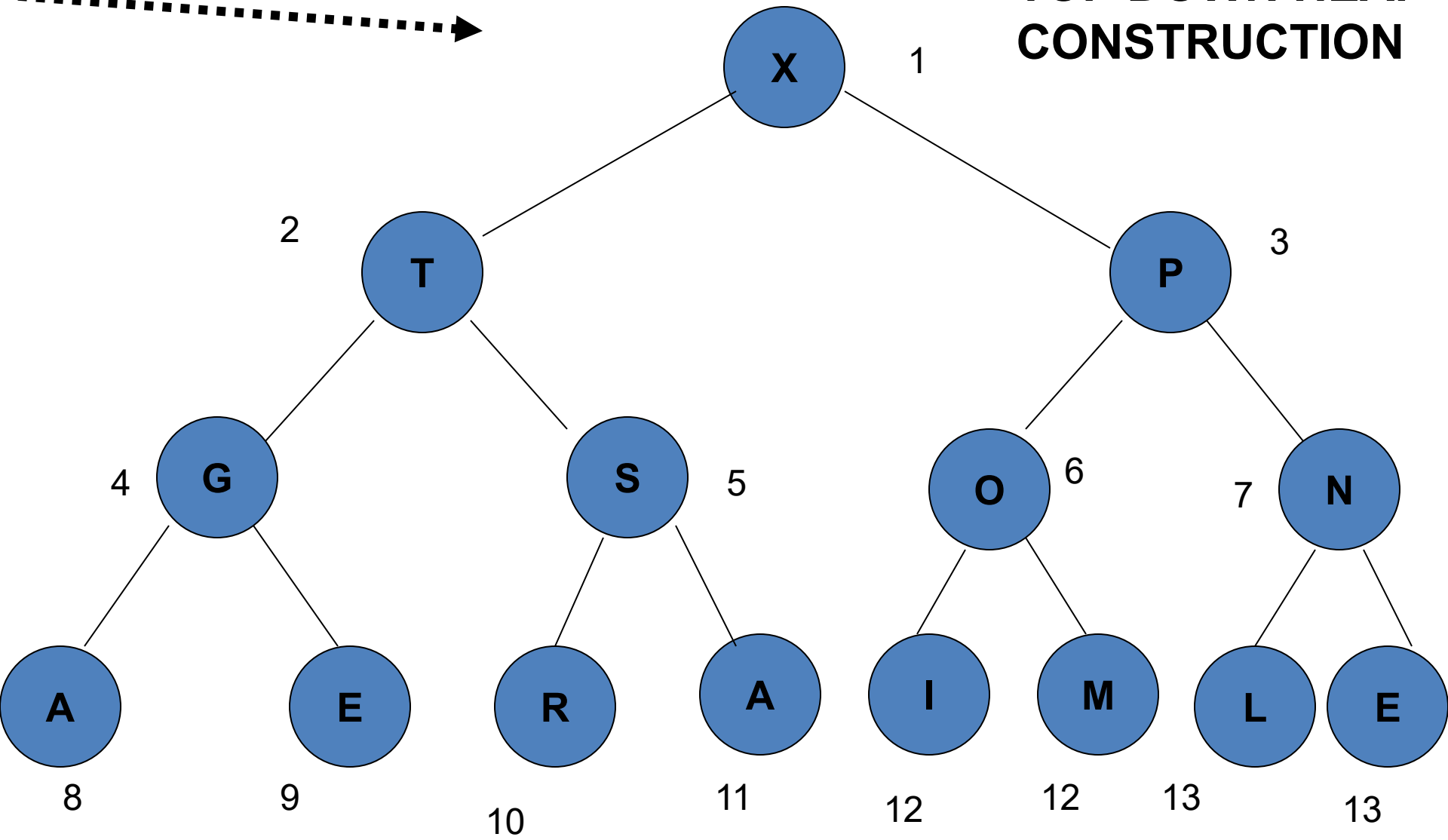

The return value is set from A[0] and then the element from A[N-1] is put into A[0] and the size of the heap is decremented, call down heap procedure to fix up the heap condition everywhere.

# Heap Sort

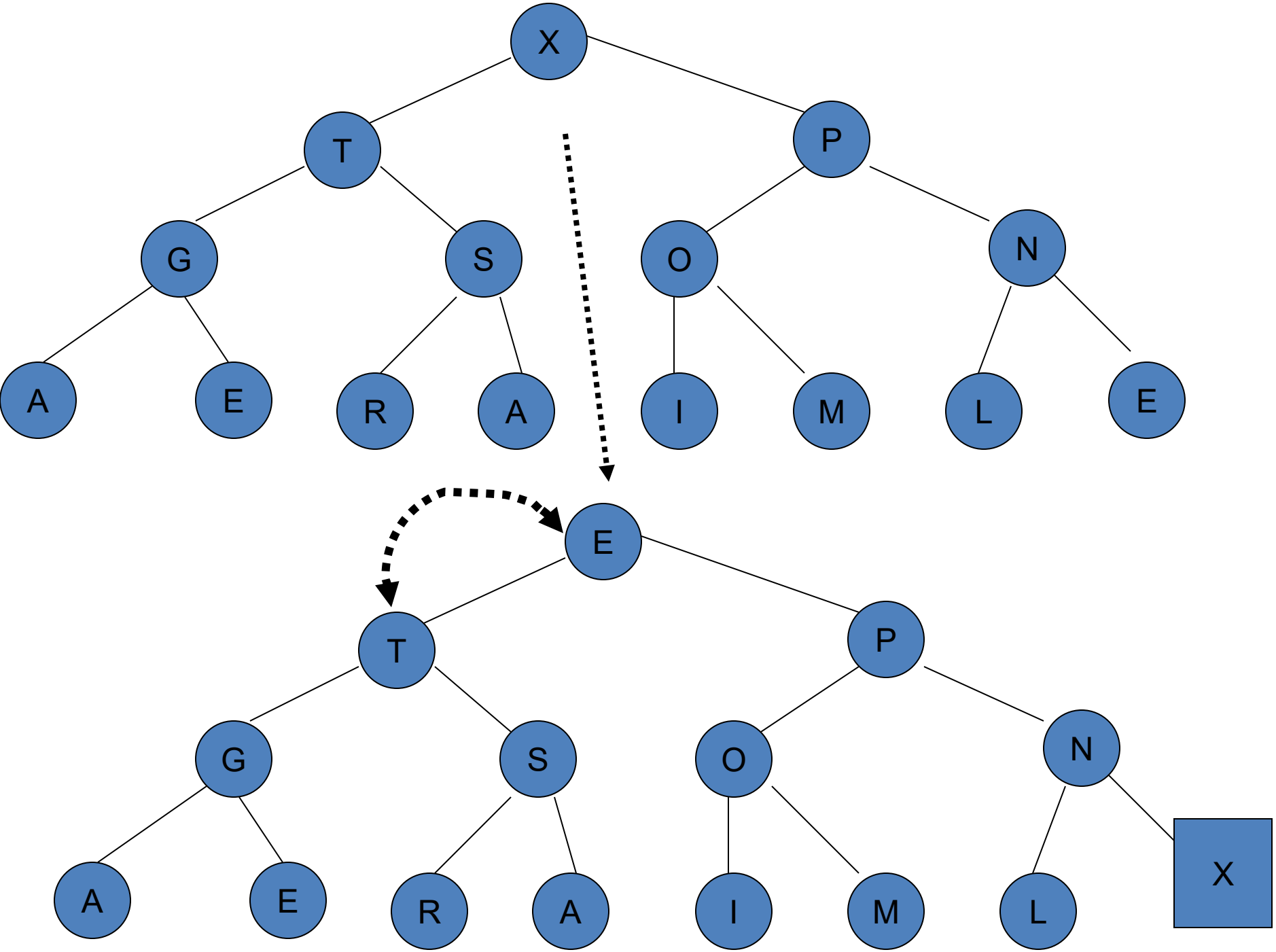
# Eg : A SORTING EXAMPLE

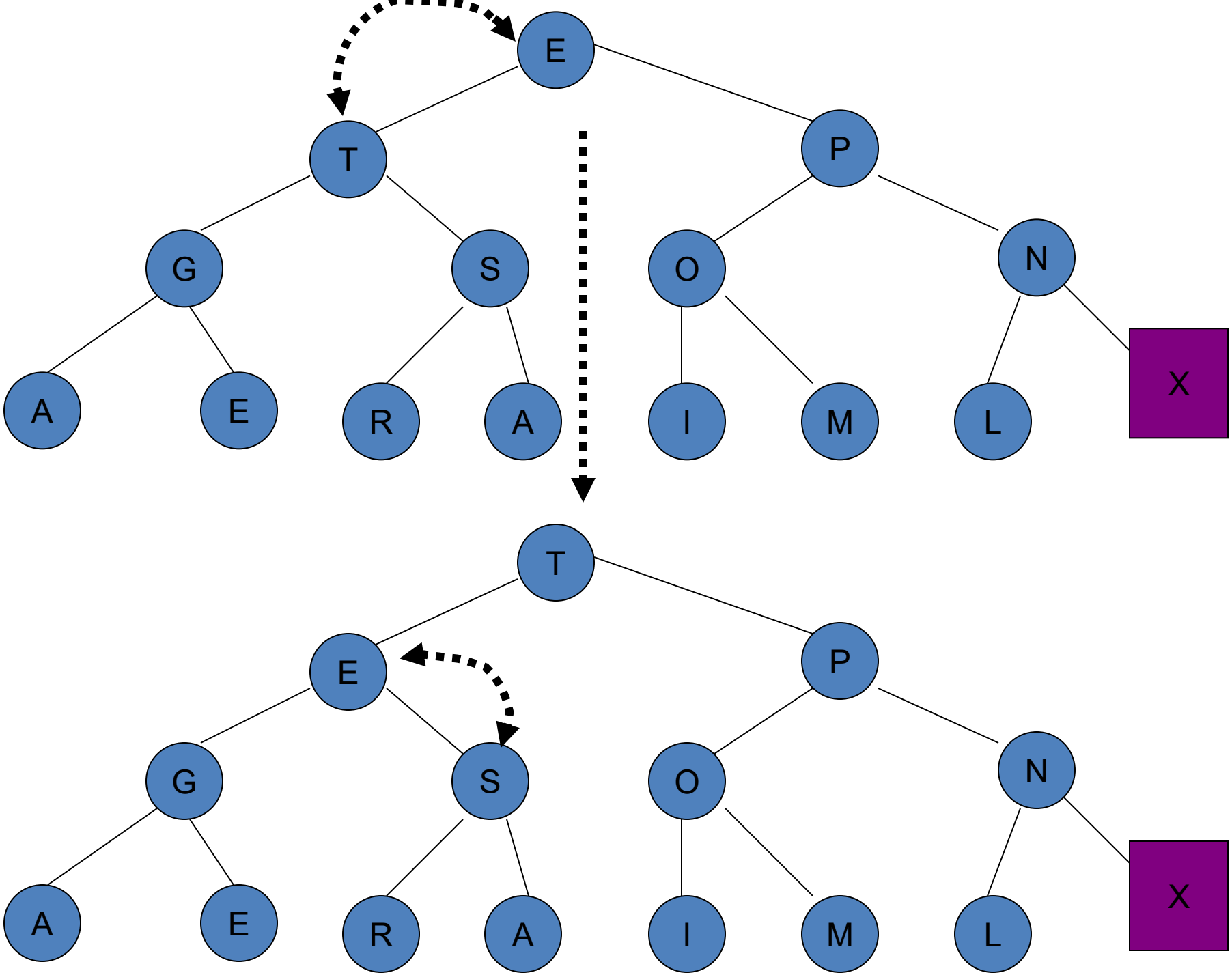


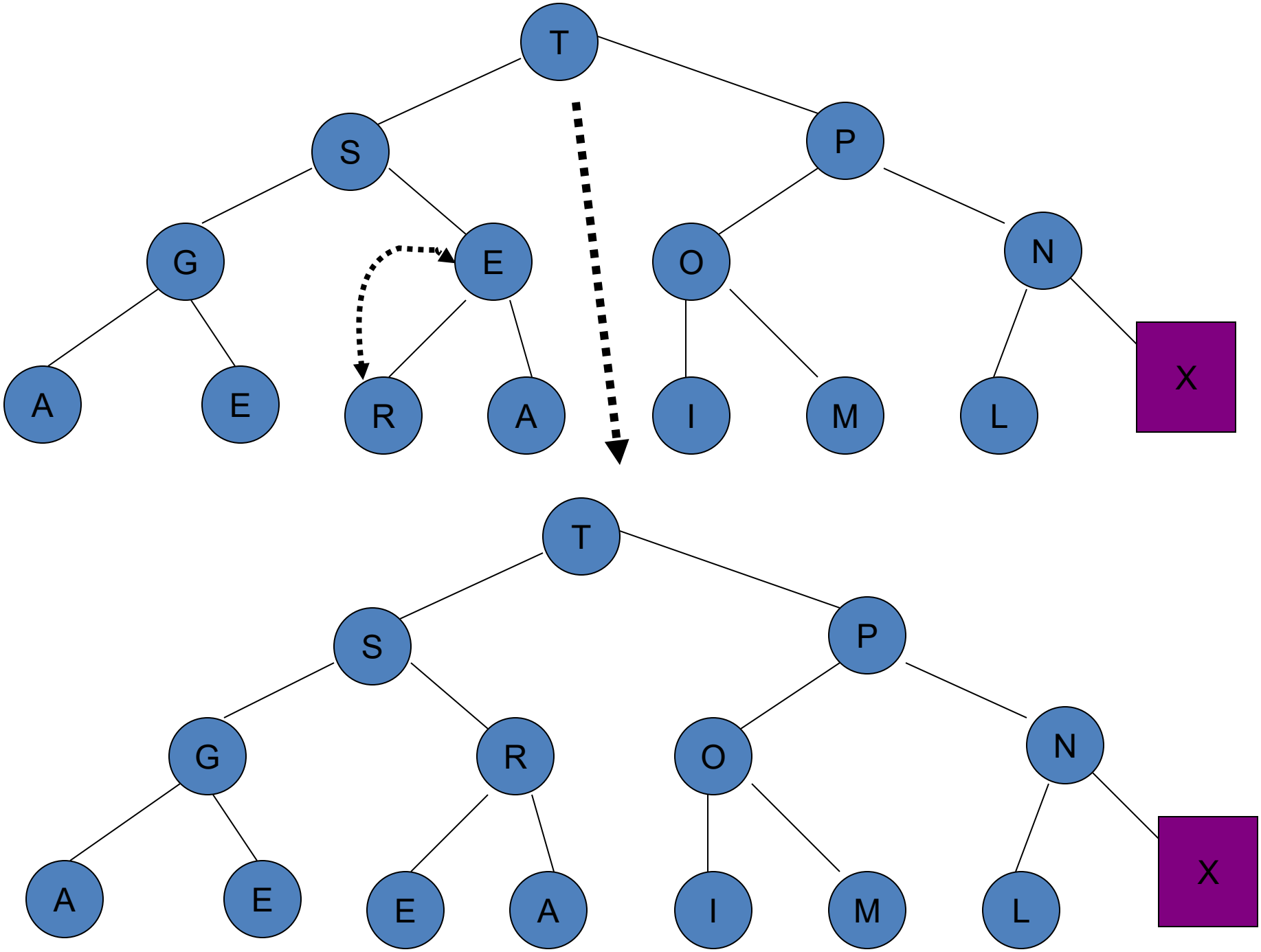
# TOP DOWN HEAP CONSTRUCTION

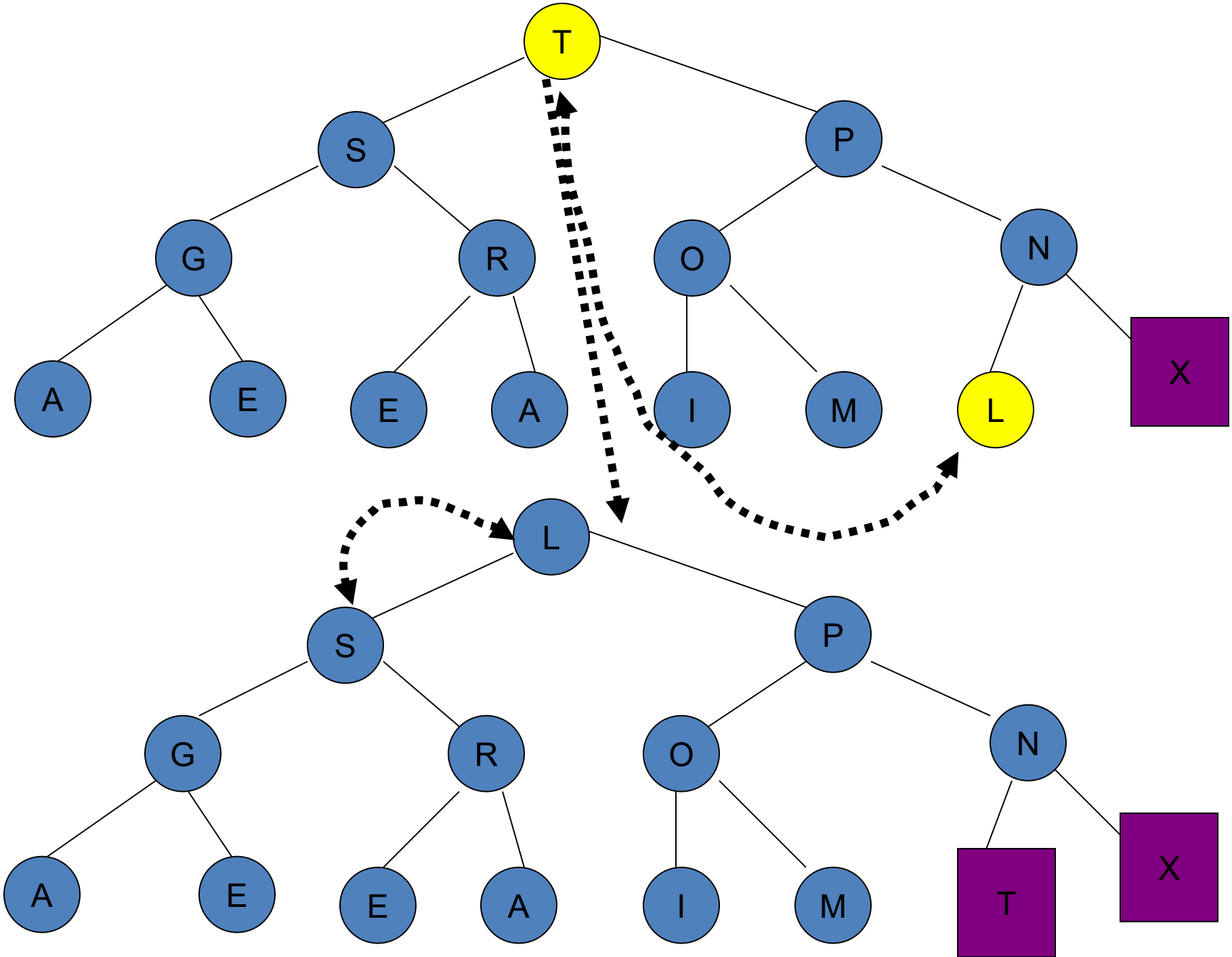


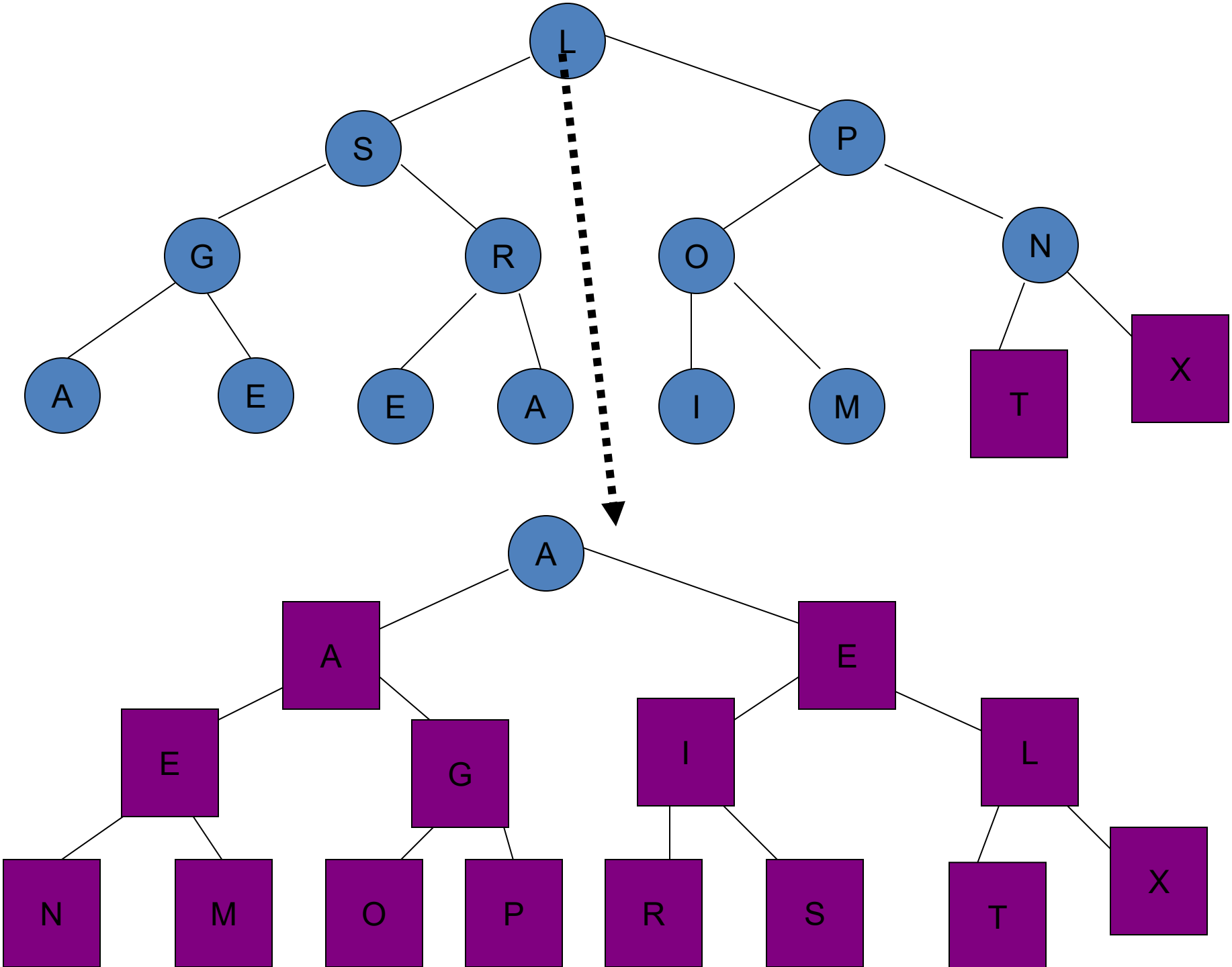

Above figure shows the heaps constructed as the keys A SORTING EXAMPLE are inserted in that order into an initially empty heap.


X
T
P
G
S
O
N
A
E
R
A
I
M
L
E
E
T
P
G
S
O
N
A
E
R
A
I
M
L
X


E
T
P
G
S
O
N
A
E
R
A
I
M
L
X
T
E
P
G
S
O
N
A
E
R
A
I
M
L
X


T
S
P
G
E
O
N
A
E
R
A
I
M
L
X
T
S
P
G
R
O
N
A
E
E
A
I
M
L
X


T
S
P
G
R
O
N
A
E
E
A
I
M
L
X
L
S
P
G
R
O
N
A
E
E
A
I
M
T
X


L
S
P
G
R
O
N
A
E
E
A
I
M
T
X
A
A
E
E
G
I
L
N
M
O
P
R
S
T
X

- 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
- A[I] A A E E G I L N M O P R S T X

- We were already noted that remove can be implemented by exchanging the first and last element and decrementing n by one and calling downheap from one.